

Sprint 1 Deliverable

University Inventory Management System

Srinidh & Jithender

Sprint Number: Sprint 1

Sprint Duration: 2 Weeks (Week 1 – Week 2)

Sprint Goal: Set up the project, design the database schema, and implement user authentication (login/logout) with role-based access control.

Team: Srinidh, Jithender

Course: Software Engineering Lab

Date: March 2026

1. Sprint Plan & Backlog

1.1 Sprint Goal

Complete the foundational setup of the University Inventory Management System. This sprint focuses on project environment setup, database design, and the implementation of the user authentication module (UC-001), enabling Admin and Verification Officers to securely log in and out of the system.

1.2 Sprint Backlog

The following user stories and tasks are selected from the product backlog for Sprint 1.

ID	User Story / Task	Priority	Est. Hrs	Status
US-01	As an Admin, I want to log in with my credentials so that I can access the system securely.	High	4	To Do
US-02	As a Verification Officer, I want to log in so that I can access the verification module.	High	2	To Do
US-03	As a user, I want to see an error message when I enter invalid credentials.	High	2	To Do
US-04	As a user, I want to log out securely so that my session is terminated.	Medium	1	To Do
T-01	Set up project repository and folder structure.	High	2	To Do
T-02	Configure Node.js backend and connect to MySQL database.	High	4	To Do
T-03	Design and implement the database schema (ER Diagram).	High	5	To Do
T-04	Implement password hashing (bcrypt) for secure storage.	High	2	To Do
T-05	Implement role-based access control (RBAC) middleware.	High	3	To Do
T-06	Create login UI page (HTML/CSS/JS).	Medium	3	To Do
T-07	Unit test authentication module.	Medium	3	To Do

1.3 Sprint Timeline

Day	Activity
Day 1–2	Project setup: repository, Node.js backend, MySQL connection
Day 3–4	Database schema design and implementation (all tables)
Day 5–6	Implement login/logout API endpoints with bcrypt and JWT/session
Day 7–8	Implement RBAC middleware; restrict routes by role
Day 9–10	Build login UI page; connect frontend to backend API
Day 11–12	Unit testing for authentication module

Day	Activity
Day 13–14	Bug fixes, code review, sprint review and retrospective

1.4 Definition of Done

- User can log in as Admin or Verification Officer using valid credentials.
- Invalid credentials display a clear error message.
- Passwords are stored as bcrypt hashes — never plain text.
- Role-based access is enforced: Admin and VO see only their allowed routes.
- Logout invalidates the session completely.
- Unit tests pass for all authentication functions.
- Database schema is created and seeded with test users.
- Code is committed to the repository with meaningful commit messages.

1.5 Sprint Risks

Risk	Likelihood	Impact	Mitigation
Delay in environment setup	Medium	High	Follow setup checklist on Day 1
Session/token implementation complexity	Medium	Medium	Use express-session or JWT library
Database connection issues	Low	High	Test connection before proceeding
Team member unavailability	Low	Medium	Share all credentials and docs

2. Database Design — ER Diagram

2.1 Overview

The database is designed using MySQL and follows a normalized relational schema. The core entities are: USER, ROOM, ITEM, QR_CODE, VERIFICATION, and VER_HISTORY. Each entity is described below, followed by the ER diagram.

2.2 Entity Descriptions

Entity	Description
USER	Stores Admin and Verification Officer accounts with hashed passwords and roles.
ROOM	Represents physical university locations (labs, classrooms, professor cabins).
ITEM	Stores all registered inventory items. Each item belongs to one room.
QR_CODE	One-to-one mapping with ITEM. Stores the generated QR data for each item.
VERIFICATION	Records each verification event — officer, item, date, and status.
VER_HISTORY	Audit log of all verification events for historical tracking and reporting.

2.3 ER Diagram

The diagram below shows all entities, their attributes (PK = Primary Key, FK = Foreign Key), and the relationships between them with cardinality.

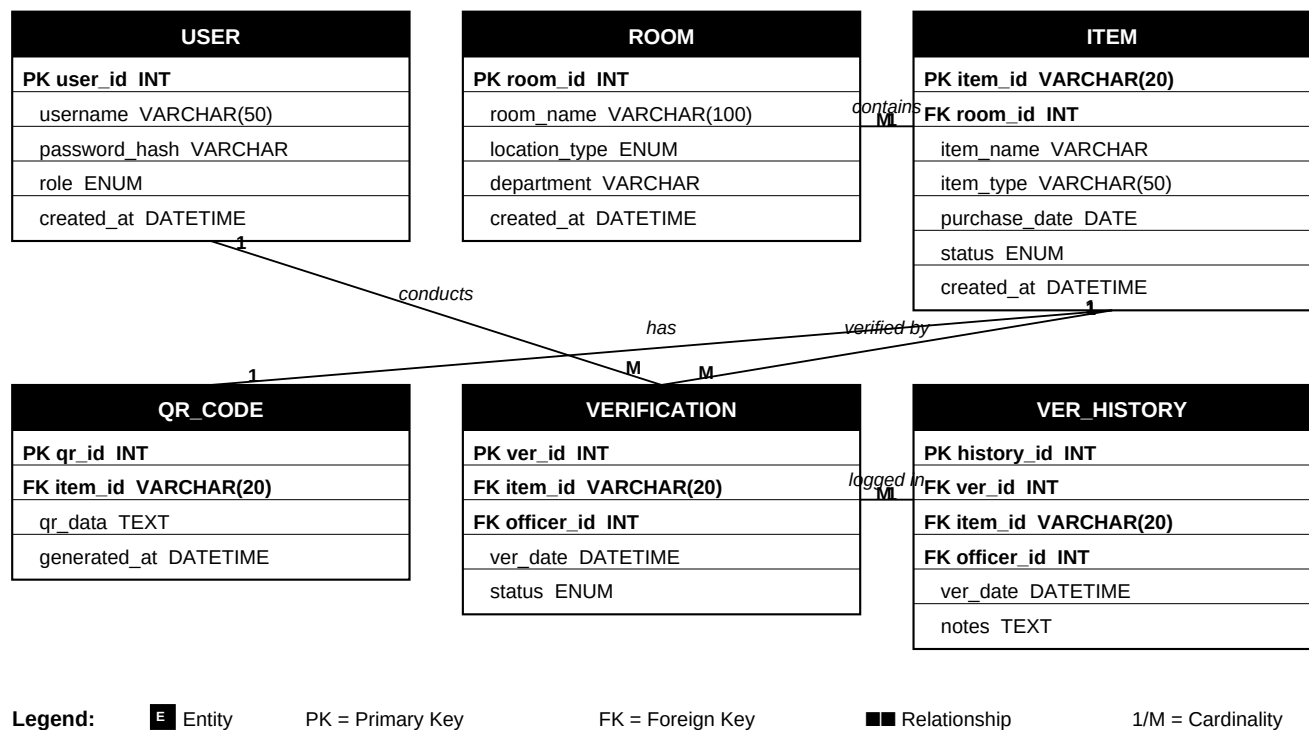


Figure 1: Entity-Relationship Diagram — University Inventory Management System

2.4 Relationships

Relationship	Type	Description
USER → VERIFICATION	1 : M	One officer can conduct many verifications
ROOM → ITEM	1 : M	One room contains many inventory items
ITEM → QR_CODE	1 : 1	Each item has exactly one QR code
ITEM → VERIFICATION	1 : M	One item can be verified multiple times over time
VERIFICATION → VER_HISTORY	1 : M	Each verification event is logged in history

2.5 Schema — SQL Table Definitions

The following SQL statements define the database schema for Sprint 1. Tables are created in dependency order (no FK referenced before its parent table).

USER table

```
CREATE TABLE USER (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) NOT NULL UNIQUE,
```

```
password_hash VARCHAR(255) NOT NULL,  
role ENUM('Admin', 'VO') NOT NULL,  
created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

ROOM table

```
CREATE TABLE ROOM (  
room_id INT PRIMARY KEY AUTO_INCREMENT,  
room_name VARCHAR(100) NOT NULL UNIQUE,  
location_type ENUM('Lab', 'Classroom', 'Cabin', 'Office') NOT NULL,  
department VARCHAR(100),  
created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

ITEM table

```
CREATE TABLE ITEM (  
item_id VARCHAR(20) PRIMARY KEY,  
room_id INT NOT NULL,  
item_name VARCHAR(100) NOT NULL,  
item_type VARCHAR(50) NOT NULL,  
purchase_date DATE,  
status ENUM('Active', 'Decommissioned') DEFAULT 'Active',  
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (room_id) REFERENCES ROOM(room_id)  
);
```

QR_CODE table

```
CREATE TABLE QR_CODE (  
qr_id INT PRIMARY KEY AUTO_INCREMENT,  
item_id VARCHAR(20) NOT NULL UNIQUE,  
qr_data TEXT NOT NULL,  
generated_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (item_id) REFERENCES ITEM(item_id)  
);
```

VERIFICATION table

```
CREATE TABLE VERIFICATION (  
ver_id INT PRIMARY KEY AUTO_INCREMENT,  
item_id VARCHAR(20) NOT NULL,
```

```

officer_id INT NOT NULL,
ver_date DATETIME DEFAULT CURRENT_TIMESTAMP,
status ENUM('Verified','Pending') DEFAULT 'Pending',
FOREIGN KEY (item_id) REFERENCES ITEM(item_id),
FOREIGN KEY (officer_id) REFERENCES USER(user_id)
);

```

VER_HISTORY table

```

CREATE TABLE VER_HISTORY (
history_id INT PRIMARY KEY AUTO_INCREMENT,
ver_id INT NOT NULL,
item_id VARCHAR(20) NOT NULL,
officer_id INT NOT NULL,
ver_date DATETIME,
notes TEXT,
FOREIGN KEY (ver_id) REFERENCES VERIFICATION(ver_id),
FOREIGN KEY (item_id) REFERENCES ITEM(item_id),
FOREIGN KEY (officer_id) REFERENCES USER(user_id)
);

```